Software Test Plan (STP)

Project: Railway Reservation System (RRS)

Team: Jackfruit Phase-1

Version: 1.0

Authors:

Yellapantula Venkata Pranav (PES2UG24CS616),

Narendra Kathare (PES2UG25CS814),

Yashash C (PES2UG24CS611)

Date: 16-09-2026

Status: Completed

1. Introduction

Purpose

This document defines the test plan for the Railway Reservation System (RRS) v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing the system described in the RRS Software Requirements Specification (SRS) v1.0 and the RRS Software Architecture Document (SAD) v1.2.

Scope

Testing covers RRS features including authentication, train search, seat availability, ticket booking, payment (simulated), PNR tracking, cancellation and refund, smart seat allocation (waitlist/RAC), and administrative train/schedule management and reporting. Real railway core infrastructure (e.g., IRCTC), real payment gateway integration, and underlying OS/network infrastructure are excluded.

References

- RRS Software Requirements Specification (SRS) v1.0, dated 05-09-2026
- RRS Software Architecture Document (SAD) v1.2, dated 14-09-2026
- IEEE 42010, OWASP guidelines (referenced in SAD Section 5.2)

Definitions

RRS (Railway Reservation System), PNR (Passenger Name Record), RAC (Reservation Against Cancellation), SRS (Software Requirements Specification), SAD (Software Architecture Document), RTM (Requirements Traceability Matrix), TC (Test Case), RBAC (Role-Based Access Control), TLS (Transport Layer Security).

2. Test Items

The following components, as defined in the SAD Component (UML) Diagram (Section 3.3), are test items for this plan:

- Web UI / CLI (presentation layer)
- Backend API (Flask HTTP/JSON layer)
- Auth Module (registration, login, session/idle timeout, lockout)
- Search Module (train search by source, destination, date)
- Booking Module (provisional booking, PNR generation, cancellation, refund)
- Payment Module and Mock Payment Module (payment processing simulation)
- Allocation Module (Confirmed/RAC/Waitlisted assignment and promotion)
- Admin Module (train/schedule management, RBAC enforcement)
- Monitoring/Reporting Module (booking, occupancy, revenue reports; audit log)
- MySQL Database (repository-style storage)

3. Features to be Tested

Features mapped to SRS requirement IDs:

- RRS-F-001 to RRS-F-003: User registration, login, account lockout (Auth Module)
- RRS-F-010, RRS-F-011: Train search and no-match handling (Search Module)
- RRS-F-020 to RRS-F-022: Seat availability display, provisional booking, field validation (Booking Module)
- RRS-F-030, RRS-F-031: Payment confirmation and release-on-failure (Payment / Mock Payment Module)
- RRS-F-040 to RRS-F-042: PNR generation/lookup, cancellation, refund calculation (Booking Module)
- RRS-F-050, RRS-F-051: Automatic seat status assignment and waitlist/RAC promotion (Allocation Module)
- RRS-F-060, RRS-F-061: Admin train/schedule management and reporting (Admin / Monitoring Module)
- RRS-NF-001 to RRS-NF-005: Performance, reliability, security/compliance, audit retention, usability
- PRJ-SR-001 to PRJ-SR-006: TLS enforcement, password hashing, server-side RBAC, session timeout, no raw card data, admin action audit logging

4. Features Not to be Tested

- Real payment gateway integration (SRS/SAD scope only covers the Mock Payment Module)
- Real railway core infrastructure integration (e.g., IRCTC/Indian Railways systems)
- Underlying OS, network infrastructure, and cloud/hosting reliability (assumed provided by the environment)
- Internal correctness of third-party libraries (Flask, MySQL connector) — assumed tested by their maintainers
- Physical hardware — none is required for this course project (SRS Section 3.2)

5. Test Approach / Strategy

Levels

- Unit tests (module-level: Auth, Search, Booking, Payment, Allocation, Admin)
- Integration tests (Booking Module ↔ Allocation Module ↔ MySQL repository interface; Payment Module ↔ Mock Payment Module)
- System tests (end-to-end RRS flow via Web UI/CLI: search → book → pay → PNR → cancel)
- Acceptance tests (UAT with course evaluator/instructor)

Types

- Functional testing (core features listed in Section 3)
- Regression testing (after each module change, e.g., allocation logic updates)
- Performance testing (response time per RRS-NF-001, load on Search/Booking modules)
- Usability testing (UI clarity, accessible status indicators per RRS-NF-005)
- Security testing (see Section 5.1)

Entry Criteria

Stable build delivered from the development team, seed test data (trains, schedules, dummy accounts) available, test environment (MySQL sandbox, mock payment module) ready.

Exit Criteria

100% of planned test cases executed, 0 critical defects open, all high-priority acceptance criteria satisfied, and the RTM (Section 13 / SRS Section 8) shows all high-priority test cases passed.

5.1 Security Validation

- Validate password hashing — no plaintext storage (PRJ-SR-002)
- TLS 1.2+ verification for all client–server connections (PRJ-SR-001)
- Server-side RBAC verification — direct API calls to admin-only endpoints rejected for non-admin roles (PRJ-SR-003)
- Session/idle timeout verification (PRJ-SR-004)
- Confirm the Mock Payment Module stores only a transaction reference and status, never raw card-like data (PRJ-SR-005)
- Confirm administrative actions are logged with user identity and timestamp (PRJ-SR-006)
- Fuzzing of input fields (booking form, login form)
- Basic penetration-style testing of authentication and admin endpoints

6. Test Environment

Hardware: Standard student laptop or development server — no special hardware required (per SRS Section 2.4/3.2)

Software: RRS backend (Python 3.x, Flask) v1.0, MySQL sandbox database, Mock Payment Module

Tools: Postman (API testing of /book and /pay endpoints), Selenium or manual testing (Web UI/CLI), JMeter (performance/response time), pytest (unit tests), GitHub Issues (defect tracking)

Test Data: Seed trains and schedules, dummy passenger accounts, mock payment card references

7. Test Schedule

Milestones:

- Test case design: 18-Sep-2026
- Environment setup: 20-Sep-2026
- Test execution start: 22-Sep-2026
- Test execution end: 29-Sep-2026
- UAT: 30-Sep-2026 to 03-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual & automated)
- Test Scripts
- Test Data
- Test Execution Logs
- Defect Reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Yashash C (PES2UG24CS611)	Prepare plan, coordinate execution
Test Engineer	Narendra Kathare (PES2UG25CS814)	Design & execute test cases, log defects
Developer	Yellapantula Venkata Pranav (PES2UG24CS616)	Support defect fixes and triage
Product Owner	Course Coordinator	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Mock payment module times out or fails, leaving a seat held indefinitely	Test the release-on-timeout path explicitly (RRS-F-031); provisional booking and held seat must return to availability
Race condition on the last available seat/berth during concurrent bookings	Include concurrency test cases against the Allocation Module (RRS-F-050); verify seat-count updates are atomic
Test environment (MySQL sandbox) downtime	Maintain a local backup MySQL instance for the team
Delay in stable build delivery from the development sub-team	Request early smoke builds; prioritize core booking flow first
Card-like data accidentally logged or persisted by the Mock Payment Module	Verify via database/log inspection that only a transaction reference and status are stored (PRJ-SR-005)

11. Assumptions & Dependencies

- The MySQL sandbox database will be stable and available for the test window
- Seed test data (trains, schedules, dummy accounts) will be provided before execution begins
- The Mock Payment Module behaves deterministically for defined test scenarios (Success/Failure per test case)
- The Flask backend and Allocation Module are feature-complete before system testing begins

12. Suspension & Resumption Criteria

Suspend testing if:

- Test environment (MySQL sandbox or mock payment module) is unavailable for more than 4 hours
- The build is too unstable and blocks more than 30% of planned test cases

Resume testing if:

- Blocking defects are resolved
- The environment is stabilized and verified with a smoke test

13. Test Case Management & Traceability

The RTM (see SRS Section 8) ensures mapping of SRS requirements to test cases. A representative excerpt is shown below; the full RTM is maintained in the SRS document and updated with Status (N/P/A) as execution proceeds.

Req ID	Requirement (short)	Module	Test case(s)
RRS-F-001 to 003	Registration, login, account lockout	Auth Module	TC-Auth-01, TC-Auth-02, TC-Auth-03
RRS-F-010, 011	Train search / no-match handling	Search Module	TC-Search-01, TC-Search-02
RRS-F-020 to 022	Seat availability, provisional booking, validation	Booking Module	TC-Book-01, TC-Book-02, TC-Book-03
RRS-F-030, 031	Payment confirmation / release on failure	Payment / Mock Payment Module	TC-Pay-01, TC-Pay-02
RRS-F-040 to 042	PNR generation/lookup, cancellation, refund	Booking Module	TC-PNR-01, TC-Cancel-01, TC-Cancel-02
RRS-F-050, 051	Auto seat status, waitlist/RAC promotion	Allocation Module	TC-Alloc-01, TC-Alloc-02
RRS-F-060, 061	Admin train/schedule management, reports	Admin / Monitoring Module	TC-Admin-01, TC-Admin-02

Req ID	Requirement (short)	Module	Test case(s)
RRS-NF-001	Response time ≤ 5s (90th percentile)	Web UI / Backend API	TC-Perf-01
PRJ-SR-001 to 006	TLS, password hashing, RBAC, session timeout, no raw card data, audit logging	Auth / Admin / Payment Module	TC-Sec-02 to TC-Sec-07

14. Test Metrics & Reporting

Metrics collected

- Percentage of test cases executed
- Percentage of test case passed / failed
- Defect density
- Defect aging
- Requirement coverage (against SRS RTM)

Reports

- Daily execution status
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead	Yashash C	
Dev Lead	Yellapantula Venkata Pranav	
Product Owner		